# NDArray Design Map

*Java N-Dimensional Array Library — Design Decisions, SIMD, Native Methods & Rationale*

Motivation: Simplex Method / Linear Programming

## 1. Core Insight — Flat Array + Index Mapper

All data lives in a single 1D primitive array. Shape and strides are separate metadata. The entire library reduces to one function:

```
f : ℤⁿ → ℤ,   flatIndex = offset + Σ (i□ × stride□)
```

**Shape** describes what the tensor looks like to the user. **Strides** describe how to find it in memory. They are intentionally decoupled.

### Row-major stride computation (C-order)

```
stride[n-1] = 1
stride[i]   = stride[i+1] × shape[i+1]
```

### Free operations via stride manipulation (no data copy)

- Transpose — swap two strides
- Slice along axis k — fix i□ as a constant, absorb into offset
- Reverse along axis — negate that stride, adjust offset

### Non-contiguous arrays — decide early

Slicing a column produces a non-contiguous view. Elements exist in data[] but are not adjacent. Options:

- Detect contiguity and fast-path System.arraycopy for matmul
- Always copy-to-contiguous before feeding compute kernels

Decide before writing multiply logic — it affects the internal API significantly.

---

## 2. The Generics Dilemma

Java generics use type erasure — at runtime NDArray<Double> and NDArray<Integer> are the same class. This means you cannot write T[] and get a primitive array. The tension is a three-way tradeoff:

| Approach | Performance | Type Safety | Generality | What You Lose |
|---|---|---|---|---|
| Option 1 — Object[] + <T> | ❌Boxing overhead | ✅Compile-time | ✅Single class | Cache perf, heap scatter |
| Option 2 — DType enum | ✅Primitive arrays | □ Runtime only | ✅Single class | Compile-time dtype checks |
| Option 3 — Subclass per dtype | ✅Primitive arrays | ✅Compile-time | □ Multiple classes | Logic duplication risk |

| | | | | |
|---|---|---|---|---|
| ✅FINAL — sealed Dtype<T> + NDArray<T> | ✅Primitive arrays | ✅Compile-time call-site | ✅Single generic class | Internal dispatch still needed |

### Why Option 1 fails for numerical work

Object[] stores pointers to heap-allocated Double objects, not contiguous doubles. A 1000×1000 matrix becomes one million scattered heap objects. Cache performance collapses entirely.

### Why Option 2 loses at the API boundary

The DType enum is runtime metadata only. The compiler cannot reject NDArray<Double>.add(NDArray<Float>) — it compiles fine and blows up at runtime.

### Why Option 3 alone is insufficient — and why it becomes the final answer

Shape/stride logic gets duplicated unless pushed into an abstract base. With an abstract base holding all shared invariants, the subclasses become clean and simple — each one only knows about its own primitive array type. This is the final design, refined by separating shared logic from type-specific logic.

---

## 3. Final Design — Abstract NDArray<T> + Typed Subclasses

The final design combines Option 2 and Option 3. The abstract base class owns everything shared — shape, strides, offset, index math, reshape, slice, transpose. Concrete subclasses own exactly one thing: their primitive array and the operations on it. No switch, no cast, no clutter inside any single class.

### 3.1 The Dtype Token — unchanged

A sealed interface carries the type parameter at compile time. Still used by factory methods and the OpsProvider — but no longer drives an internal switch:

```
public sealed interface Dtype<T extends Number>
    permits Dtype.F64, Dtype.F32, Dtype.I32 {

  Dtype<Double>  F64 = new F64();
  Dtype<Float>   F32 = new F32();
  Dtype<Integer> I32 = new I32();

  record F64() implements Dtype<Double>  {}
  record F32() implements Dtype<Float>   {}
  record I32() implements Dtype<Integer> {}
}
```

### 3.2 NDArray<T> — the abstract base

Owns all shared invariants. No data array — no knowledge of what type lives below. Shape/stride constructor is called via super() and is the only place this logic lives:

```
public abstract class NDArray<T extends Number> {
  protected final int[]   shape;
  protected final int[]   strides;
  protected final int     offset;
```

```
  protected final boolean contiguous;

  protected NDArray(int[] shape, int[] strides, int offset) {
    this.shape      = shape;
    this.strides    = strides;
    this.offset     = offset;
    this.contiguous = computeContiguity(shape, strides);
  }

  // shared — written once, never repeated
  protected int flatIndex(int[] indices) { ... }
  public NDArray<T> reshape(int... newShape) { ... }
  public NDArray<T> slice(int axis, int index) { ... }
  public NDArray<T> transpose() { ... }
  public int[] shape() { return shape.clone(); }
  public int   rank()  { return shape.length; }

  // abstract hooks — each subclass fills in
  public abstract NDArray<T> add(NDArray<T> other);
  public abstract NDArray<T> matmul(NDArray<T> other);
  public abstract NDArray<T> zerosLike();
  public abstract Dtype<T>   dtype();
}
```

### 3.3 Concrete subclasses — one per dtype

Each subclass is simple, readable, and self-contained. The subclass owns the primitive array and delegates every operation to its operator. No switch, no Object cast, no unrelated branches anywhere:

```
public final class F64Array extends NDArray<Double> {
  final double[] data;   // package-private — operator needs direct access

  // static final — resolved once at class load, JIT devirtualizes to zero cost
  private static final F64Operator OP = OpsProvider.load().f64();

  F64Array(double[] data, int[] shape, int[] strides, int offset) {
    super(shape, strides, offset);
    this.data = data;
  }

  @Override public Dtype<Double> dtype() { return Dtype.F64; }

  @Override
  public F64Array add(NDArray<Double> other) {
    return OP.add(this, (F64Array) other);  // one line — no logic here
  }

  @Override
```

```
  public F64Array matmul(NDArray<Double> other) {
    return OP.matmul(this, (F64Array) other);
  }

  @Override
  public F64Array zerosLike() {
    return new F64Array(new double[data.length], shape, strides, 0);
  }
}
```

F32Array and I32Array are identical in structure — each holds its own static final operator. The class body is entirely routing — no arithmetic logic lives here at all.

## 3.4 The Operator interface — one per dtype

The operator carries the backend implementation. Each method is a pure function from two arrays to a result. No state, no array ownership — just the computation:

```
public interface F64Operator {
  F64Array add    (F64Array a, F64Array b);
  F64Array sub    (F64Array a, F64Array b);
  F64Array mul    (F64Array a, F64Array b);   // element-wise
  F64Array matmul (F64Array a, F64Array b);
  double   dot    (F64Array a, F64Array b);
  F64Array map    (F64Array a, DoubleUnaryOperator fn);
}

// Parallel interfaces for the other dtypes
public interface F32Operator { ... }
public interface I32Operator { ... }
```

## 3.5 Operator implementations

Each implementation is a small, focused class. ScalarF64Operator is the always-correct baseline. SimdF64Operator and CudaF64Operator are drop-in replacements — same interface, different body:

```
// ScalarF64Operator — plain loops, always correct
public final class ScalarF64Operator implements F64Operator {
  @Override
  public F64Array add(F64Array a, F64Array b) {
    double[] result = new double[a.data.length];
    for (int i = 0; i < result.length; i++)
      result[i] = a.data[i] + b.data[i];
    return new F64Array(result, a.shape, a.strides, 0);
  }
  // other ops follow same pattern
}

// SimdF64Operator — Vector API fast path
public final class SimdF64Operator implements F64Operator {
  private static final VectorSpecies<Double> SPECIES =
      DoubleVector.SPECIES_PREFERRED;
```

```
  private static final F64Operator SCALAR = new ScalarF64Operator();

  @Override
  public F64Array add(F64Array a, F64Array b) {
    if (!a.contiguous) return SCALAR.add(a, b); // non-contiguous fallback
    double[] result = new double[a.data.length];
    int i = 0, upper = SPECIES.loopBound(a.data.length);
    for (; i < upper; i += SPECIES.length())
      DoubleVector.fromArray(SPECIES, a.data, i)
        .add(DoubleVector.fromArray(SPECIES, b.data, i))
        .intoArray(result, i);
    for (; i < a.data.length; i++) result[i] = a.data[i] + b.data[i];
    return new F64Array(result, a.shape, a.strides, 0);
  }
}

// CudaF64Operator — JNI to CUDA kernels
public final class CudaF64Operator implements F64Operator {
  @Override
  public F64Array matmul(F64Array a, F64Array b) { /* JNI call */ }
  // delegates non-GPU ops to SimdF64Operator
  private static final F64Operator SIMD = new SimdF64Operator();
  @Override
  public F64Array add(F64Array a, F64Array b) { return SIMD.add(a, b); }
}
```

### 3.6 Factory — Class<T> never exposed to caller

Factory methods delegate to the OpsProvider which resolves the richest available operator. The caller sees only NDArray<Double>:

```
public static <T extends Number> NDArray<T> of(Dtype<T> dtype, int... shape) {
  return PROVIDER.create(dtype, shape);
}

NDArray<Double> m = NDArray.of(Dtype.F64, 3, 4);
NDArray<Double> r = m.add(other);  // routes to OP.add — zero overhead
```

---

## 4. What <T extends Number> Actually Buys You

### Call-site dtype enforcement — two levels

The generic parameter enforces dtype at the API boundary. Passing F32Array where F64Array is expected is a compile error:

```
NDArray<Double> a = NDArray.of(Dtype.F64, 3, 4);
NDArray<Float>  b = NDArray.of(Dtype.F32, 3, 4);
a.add(b);  // ← compile error — NDArray<Float> not NDArray<Double>
```

### Return type flows without casting

```
NDArray<Double> result = matrix.add(other);  // no cast at call site
```

### Generic algorithms written once in the base

```
public static <T extends Number> NDArray<T> elementWise(
    NDArray<T> a, NDArray<T> b, BinaryOperator<T> op) { ... }
```

### zerosLike preserves dtype automatically

```
public NDArray<T> zerosLike() { ... }  // T preserved through the hierarchy
```

### Where <T> does NOT help

The one cast per method in each subclass — (F64Array) other — is unavoidable. It is localized to one line per method inside the subclass. The operator implementations access a.data and b.data directly with no cast at all, since they receive the concrete type.

---

## 5. Architecture — Two Orthogonal Axes

The design has two completely independent axes that compose cleanly. Dtype subclasses handle WHAT primitive type is stored. Operator implementations handle HOW operations are computed. Neither knows about the other.

### Axis 1 — Dtype hierarchy (readability, type safety)

- NDArray<T> (abstract) — shape, strides, offset, flatIndex, reshape, slice, transpose. No data. No arithmetic.
- F64Array / F32Array / I32Array — own the primitive array. Route every op to their static final operator. Body is pure delegation — no logic.

### Axis 2 — Operator hierarchy (backend, performance)

- F64Operator / F32Operator / I32Operator — interfaces defining the computation contract.
- ScalarXxxOperator — plain loops. Always correct. The guaranteed fallback.
- SimdXxxOperator — Vector API fast path. Falls back to scalar when non-contiguous.
- CudaF64Operator — JNI to CUDA kernels for matmul. Delegates non-GPU ops to SIMD.

### How they compose

F64Array holds a static final F64Operator. Which concrete operator it holds is decided once at class load time by OpsProvider. The JIT sees a monomorphic static final field and devirtualizes the call — zero overhead at runtime:

| | ScalarXxxOperator | SimdXxxOperator | CudaF64Operator |
|---|---|---|---|
| F64Array | plain double[] loops | Vector API, AVX-512 | CUDA matmul via JNI |
| F32Array | plain float[] loops | Vector API, 16 lanes | — (not planned) |
| I32Array | plain int[] loops | Vector API, 16 lanes | — (not planned) |

*Each cell is a valid combination. OpsProvider selects the operator column at class load time. Dtype subclass is the row — chosen by the user via Dtype.F64 / F32 / I32.*

### Mutability rule — decide early

Make the backing array mutable (efficient in-place pivot ops). Make shape and stride metadata immutable after construction. Shape/stride constructor lives in NDArray abstract base only — never duplicated in subclasses.

### Contiguity flag — lives in the abstract base

Computed once at construction, stored as a final field, read by operators. The operator is responsible for its own fallback — it holds a reference to ScalarOperator and delegates when non-contiguous:

```
// in NDArray abstract base — computed once, inherited by all
protected final boolean contiguous;

private static boolean computeContiguity(int[] shape, int[] strides) {
  int expected = 1;
  for (int i = shape.length - 1; i >= 0; i--) {
    if (strides[i] != expected) return false;
    expected *= shape[i];
  }
  return true;
}
```

---

## 6. SIMD — Vector API (jdk.incubator.vector)

The Vector API (JEP 338, incubating since Java 16) maps directly to CPU SIMD registers — AVX2/AVX-512 on x86, NEON on ARM. This is not java.util.Vector. It is hardware-level parallelism inside a single core.

### 6.1 Why the flat array design enables this

SIMD requires contiguous memory. Object[] with boxed Doubles is scattered heap pointers — the Vector API cannot touch it. The flat double[] backing array chosen from the beginning is exactly what DoubleVector.fromArray requires. The entire design was pointing here.

### 6.2 VectorSpecies — one per dtype

SPECIES_PREFERRED picks the widest register your CPU supports at runtime. Each dtype gets its own species — float gets twice the lanes of double on the same hardware:

```
import jdk.incubator.vector.*;

static final VectorSpecies<Double>  F64_SPECIES = DoubleVector.SPECIES_PREFERRED;
static final VectorSpecies<Float>   F32_SPECIES = FloatVector.SPECIES_PREFERRED;
static final VectorSpecies<Integer> I32_SPECIES = IntVector.SPECIES_PREFERRED;
// AVX2:   F64 = 4 lanes,  F32 = 8 lanes,  I32 = 8 lanes
// AVX-512: F64 = 8 lanes, F32 = 16 lanes, I32 = 16 lanes
```

### 6.3 The vectorized loop pattern

Lives entirely inside SimdF64Operator — a focused class with one job. Accesses a.data and b.data directly since it receives the concrete F64Array type:

```
// SimdF64Operator.java — self-contained, nothing else in view
@Override
public F64Array add(F64Array a, F64Array b) {
  if (!a.contiguous) return SCALAR.add(a, b); // delegate to scalar
  double[] result = new double[a.data.length];
  int i = 0, upper = SPECIES.loopBound(a.data.length);

  for (; i < upper; i += SPECIES.length()) {
    DoubleVector.fromArray(SPECIES, a.data, i)
      .add(DoubleVector.fromArray(SPECIES, b.data, i))
      .intoArray(result, i);
  }
  for (; i < a.data.length; i++) result[i] = a.data[i] + b.data[i];
  return new F64Array(result, a.shape, a.strides, 0);
}
```

## 6.4 Dispatch strategy — operator holds its own fallback

There is no flag in F64Array. No switch. The operator itself is responsible for the contiguity check and holds a scalar reference for non-contiguous fallback. F64Array.add is one line:

```
// F64Array — pure routing, zero logic
public F64Array add(NDArray<Double> other) {
  return OP.add(this, (F64Array) other);
}

// SimdF64Operator — owns the decision
private static final F64Operator SCALAR = new ScalarF64Operator();
public F64Array add(F64Array a, F64Array b) {
  if (!a.contiguous) return SCALAR.add(a, b);
  // ... SIMD path
}
```

## 6.5 Dot product — reduction with SIMD

Dot product needs a reduction step after the vectorized multiply. DoubleVector.reduceLanes handles this — the result is a single scalar:

```
public double dot(NDArray<Double> other) {
  double[] a = (double[]) this.data;
  double[] b = (double[]) other.data;
  DoubleVector acc = DoubleVector.zero(F64_SPECIES);
  int i = 0, upper = F64_SPECIES.loopBound(a.length);

  for (; i < upper; i += F64_SPECIES.length())
    acc = DoubleVector.fromArray(F64_SPECIES, a, i)
              .fma(DoubleVector.fromArray(F64_SPECIES, b, i), acc);
  double result = acc.reduceLanes(VectorOperators.ADD);
  for (; i < a.length; i++) result += a[i] * b[i];
```

```
  return result;
}
```

fma = fused multiply-add. One instruction computes a*b+c with a single rounding step — faster and more numerically stable than separate multiply then add.

### 6.6 Module flag required

The Vector API is still incubating. You must pass this flag at both compile and runtime:

```
javac --add-modules jdk.incubator.vector ...
java  --add-modules jdk.incubator.vector ...
```

---

## 7. Native Methods — JNI and Project Panama

For the absolute performance ceiling — particularly matrix multiplication — calling native code bypasses JIT uncertainty entirely and gives access to 40 years of hand-tuned BLAS optimizations.

### 7.1 JNI — direct AVX intrinsics

JNI lets you write a C kernel with raw AVX intrinsics, called from Java. GetPrimitiveArrayCritical gives a direct pointer to the Java array's backing memory with no copy:

```
// Java side
private static native void addF64Native(double[] a, double[] b, double[] out, int
len);
static { System.loadLibrary("ndarray_native"); }


// C side — raw AVX-512
#include <immintrin.h>
JNIEXPORT void JNICALL Java_NDArray_addF64Native(
    JNIEnv *env, jclass cls,
    jdoubleArray ja, jdoubleArray jb, jdoubleArray jout, jint len) {
  double *a   = (*env)->GetPrimitiveArrayCritical(env, ja,   0);
  double *b   = (*env)->GetPrimitiveArrayCritical(env, jb,   0);
  double *out = (*env)->GetPrimitiveArrayCritical(env, jout, 0);
  for (int i = 0; i <= len-8; i += 8) {
    __m512d va = _mm512_loadu_pd(&a[i]);
    __m512d vb = _mm512_loadu_pd(&b[i]);
    _mm512_storeu_pd(&out[i], _mm512_add_pd(va, vb));
  }
  (*env)->ReleasePrimitiveArrayCritical(env, ja,   a,   JNI_ABORT);
  (*env)->ReleasePrimitiveArrayCritical(env, jb,   b,   JNI_ABORT);
  (*env)->ReleasePrimitiveArrayCritical(env, jout, out, 0);
}
```

### 7.2 Project Panama — call OpenBLAS directly

Panama (graduated Java 22) eliminates all JNI boilerplate. You call OpenBLAS dgemm directly from Java with no C glue code whatsoever. dgemm is the most optimized matrix multiply ever written:

```
import java.lang.foreign.*;
import java.lang.invoke.*;
```

```
MethodHandle dgemm = linker.downcallHandle(
    lookup.find("cblas_dgemm").orElseThrow(),
    FunctionDescriptor.ofVoid(
        ValueLayout.JAVA_INT,    // order (row-major = 101)
        ValueLayout.JAVA_INT,    // transA
        ValueLayout.JAVA_INT,    // transB
        ValueLayout.JAVA_INT,    // M
        ValueLayout.JAVA_INT,    // N
        ValueLayout.JAVA_INT,    // K
        ValueLayout.JAVA_DOUBLE, // alpha
        ValueLayout.ADDRESS,     // A pointer
        ValueLayout.JAVA_INT,    // lda
        ValueLayout.ADDRESS,     // B pointer
        ValueLayout.JAVA_INT,    // ldb
        ValueLayout.JAVA_DOUBLE, // beta
        ValueLayout.ADDRESS,     // C pointer
        ValueLayout.JAVA_INT     // ldc
    )
);
```

### 7.3 Activation flag — layered fallback

Native ops are optional. A single system property gates them at startup. The scalar path is always the final fallback — the library runs correctly on any JVM regardless of native library availability:

```
private static final boolean USE_NATIVE =
    Boolean.getBoolean("ndarray.native") && NativeLoader.isAvailable();

public NDArray<T> add(NDArray<T> other) {
  if (USE_NATIVE)              return nativeAdd(other);
  if (this.contiguous)         return simdAdd(other);
  return scalarAdd(other);
}
```

---

## 8. Multithreading Strategy

Threading multiplies the gains from SIMD. On 16 cores with AVX-512: 8 doubles/instruction × 16 cores = 128x over scalar per cycle. Use ForkJoinPool with RecursiveAction, sequential threshold ~64 rows:

| Operation | Worth Parallelizing | Reason |
|---|---|---|
| Matrix multiplication | ✅Yes | $O(n^3)$, rows are independent |
| Simplex pivot row ops | ✅Yes | Each non-pivot row update is independent |
| Element-wise add/sub | ⚠ Large n only | Thread overhead dominates for small arrays |

| | | |
|---|---|---|
| Dot product | ⚠ Careful | Needs reduction step — use ForkJoinPool or LongAdder |

---

## 9. Realistic Performance Stack

Each optimization layer compounds. The gap between naive Java and the full stack is ~200x — entirely under your control:

| Optimization Layer | Approx. Speedup | Note |
|---|---|---|
| Pure Java naive double loops | 1x (baseline) | Correct but no optimization |
| + Cache-aware tiling | ~5-10x | Highest single impact for matmul. Free — pure algorithm. |
| + SIMD Vector API (AVX-512) | ~8x per core | 8 doubles/instruction. Requires contiguous array. |
| + ForkJoinPool (16 cores) | ~100x cumulative | Rows independent. Threshold at n=64 to avoid overhead. |
| + JNI with raw AVX intrinsics | ~120-150x | Bypasses JIT uncertainty. Direct hardware control. |
| + Panama → OpenBLAS dgemm | ~200-400x | 40 years of hardware-specific tuning. No C glue needed. |
| vs SciPy (NumPy + BLAS) reference | ~40-60% | Remaining gap: manual prefetch, micro-arch tricks unavailable in JVM. |

### Memory bandwidth — the real ceiling

Compute throughput is not the bottleneck for element-wise operations. RAM bandwidth is:

```
DDR5 bandwidth  : ~50 GB/s
50 GB/s / 8 bytes per double = 6.25 GFLOPS memory bound
CPU compute peak (AVX-512, 16 cores) : ~448 GFLOPS
```

For element-wise add/sub you hit the RAM ceiling long before the compute ceiling. Matrix multiplication is the exception — it reuses each element O(n) times, making it compute-bound. This is why tiling matters: keeping submatrices in L1 cache across the reuse window.

### Cache tiling — the single highest-impact optimization

Naive matmul walks columns of B — stride = n doubles = cache miss every step. Tiled matmul keeps TILE×TILE submatrices hot in L1 (tune TILE to ~32KB / 8 bytes = 64):

```
for (int i = 0; i < n; i += TILE)
  for (int j = 0; j < n; j += TILE)
    for (int k = 0; k < n; k += TILE)
```

```
            multiplyTile(a, b, c, i, j, k, TILE);
        // each TILE×TILE block reused n/TILE times before eviction
```

This alone gives 5-10x over naive on large matrices, before SIMD or threading.

---

# 10. Operator Dispatch — Dynamic Implementation Selection

To allow the implementation of every operation to vary at startup (scalar, SIMD, BLAS, CUDA) without any cost in hot code, JavaLin uses a layered dispatch architecture. The key insight is that the choice is made once and becomes structurally constant — the JIT sees a monomorphic call site and eliminates all indirection.

## 10.1 The Performance Concern — Quantified

Virtual dispatch overhead depends entirely on how many implementations the JIT sees at a given call site:

| Call Type | Approx. Cost | JIT Outcome | When It Applies |
|---|---|---|---|
| Direct static call | ~1 ns | Inlined | No polymorphism at all |
| Devirtualized virtual call | ~1 ns | Inlined | static final, one impl ever loaded |
| Monomorphic virtual call | ~2 ns | Usually inlined | One impl in practice, JIT observes |
| Bimorphic virtual call | ~3 ns | Partial inline | Two impls observed at call site |
| Megamorphic virtual call | ~10 ns | vtable fallback | 3+ impls — avoid in hot loops |
| MethodHandle (static final) | ~1 ns | Fully inlined | Constant target — same as direct call |

The goal is to stay in the top two rows. static final INSTANCE set once means the JIT always sees exactly one concrete type — devirtualized, zero overhead.

## 10.2 static final — The JIT's Escape Hatch

static final fields are treated as compile-time constants by the JIT. Dead branches disappear from bytecode entirely after JIT compilation. The flag check has genuinely zero runtime cost:

```
        // This branch is eliminated entirely by JIT after warmup
        static final boolean USE_CUDA = Boolean.getBoolean("javalin.cuda");

        // This becomes a direct static call — no vtable, no null check
        static final Ops<?> INSTANCE = Ops.load();
```

## 10.3 The OpsProvider SPI

The Strategy pattern is expressed as a Service Provider Interface. JavaLin core defines the contract. Implementations live in separate JARs — core never imports them:

```
        // javalin-core — the contract only
```

```
public interface OpsProvider {
  <T extends Number> Ops<T> ops(Dtype<T> dtype);
  int     priority();      // higher wins
  boolean isAvailable();   // checks hardware at runtime

  static OpsProvider load() {
    return ServiceLoader.load(OpsProvider.class)
      .stream()
      .map(ServiceLoader.Provider::get)
      .filter(OpsProvider::isAvailable)
      .max(Comparator.comparingInt(OpsProvider::priority))
      .orElse(new ScalarOpsProvider());  // always correct fallback
  }
}
```

## 10.4 NDArray delegates to the resolved provider

NDArray calls through the provider — one indirection that the JIT devirtualizes because PROVIDER is static final and monomorphic:

```
public class NDArray<T extends Number> {
  private static final OpsProvider PROVIDER = OpsProvider.load();

  public NDArray<T> add(NDArray<T> other) {
    return PROVIDER.ops(this.dtype).add(this, other);
  }
}
```

## 10.5 MethodHandle — zero-overhead alternative

For finer control, MethodHandle stored in a static final field achieves the same JIT guarantee. The JIT recognizes a constant call target and inlines the callee directly — the same mechanism Java Lambda expressions use internally:

```
static final MethodHandle ADD_HANDLE = selectAddHandle();

private static MethodHandle selectAddHandle() {
  if (CudaRuntime.isAvailable())
    return lookup.findStatic(CudaOps.class,  "add", MT);
  if (SimdOps.isAvailable())
    return lookup.findStatic(SimdOps.class,  "add", MT);
  return   lookup.findStatic(ScalarOps.class,"add", MT);
}
```

Unlike interface dispatch, a constant MethodHandle has a provably single target. Zero vtable lookup, zero branch, fully inlined by C2.

## 10.6 Module architecture — the full picture

Each implementation is a separate Maven/Gradle module with its own JAR. JavaLin core has zero compile-time dependency on any of them. Resolution is entirely runtime via ServiceLoader:

```
javalin-core          priority  0  — ScalarOps, always correct
javalin-simd          priority 10  — Vector API, Java 16+
```

```
javalin-native        priority 50  — JNI + OpenBLAS/MKL
javalin-cuda          priority 100 — CUDA via JNI, GPU required

// Resolution at startup — automatic, silent, once:
CUDA available?   → CudaOpsProvider   wins
BLAS available?   → NativeOpsProvider wins
Java 16+?         → SimdOpsProvider   wins
fallback          → ScalarOpsProvider (always present)
```

### 10.7 Future extension — CUDA

Adding CUDA requires no changes to JavaLin core whatsoever. A new module implements OpsProvider, calls CUDA kernels via JNI, and declares itself in module-info.java. Drop the JAR on the classpath and it activates:

```
// javalin-cuda/module-info.java
module javalin.cuda {
  requires javalin.core;
  provides javalin.spi.OpsProvider
      with javalin.cuda.CudaOpsProvider;
}

public class CudaOpsProvider implements OpsProvider {
  public int     priority()    { return 100; }
  public boolean isAvailable() { return CudaRuntime.deviceCount() > 0; }
  public <T extends Number> Ops<T> ops(Dtype<T> dtype) {
    return new CudaOps<>();
  }
}
```

---

## 11. Java Limitations & Future Notes

### Type erasure

Java generics are compile-time only. NDArray<Double> and NDArray<Integer> are the same class at runtime. The sealed Dtype<T> token is how we reify the type parameter without boxing.

### No primitive generics

You cannot write NDArray<double> — only NDArray<Double>. The sealed switch pattern is the workaround: the generic layer speaks to the user, the switch speaks to the machine.

### JIT warmup cost

The JVM needs ~10,000 iterations before emitting optimized machine code. For a solver running a few large matrix ops and exiting, you pay warmup cost every run. GraalVM native image (AOT compilation) eliminates this but loses some peak JIT optimizations.

### No manual prefetch

C lets you call __builtin_prefetch to tell the CPU to load memory before you need it. Java has no equivalent. The JIT sometimes inserts prefetch instructions but not reliably. This is a real but small gap vs hand-tuned BLAS.

### Project Valhalla

Future Java versions will introduce primitive types and value generics, eventually allowing NDArray<double> with zero overhead. The sealed Dtype<T> design above will be easy to migrate when Valhalla ships — the switch dispatch maps directly to what Valhalla's specialization will generate.

### How this compares to real libraries

- NumPy (Python/C) — dtype is runtime metadata, no compile-time safety. Calls MKL/OpenBLAS for matmul. Identical architecture to Layer 3 of your design.
- Eigen (C++) — true generics, zero overhead, full safety. C++ templates generate separate code per type — what Valhalla will eventually give Java.
- ND4J (Java) — one INDArray interface, dtype at runtime, off-heap backing buffer. No SIMD Vector API — predates it.
- TensorFlow Java — Panama-style native calls to XLA kernels. Closest to your Layer 3 (Panama → BLAS) approach.

---